

**KODIGO ACADEMIA BOOTCAMP MINED**

**SAN FRANCISCO GOTERA, MORAZAN EL SALVADOR**

**TRANSFORMACIÓN DIGITAL PARA LA DOCENCIA TÉCNICA**



**TEMA: API REST PROFESIONAL UTILIZANDO AUTENTICACIÓN JWT**

**GRUPO N°: 3**

**ESTUDIANTE: JOSÉ PABLO COLATO MENDOZA**

**MODULO: TRANSFORMACIÓN DIGITAL**

**TUTOR/DOCENTE: FERNANDO JOSE AGUILAR RIVAS**

**PERIODO/AÑO: 012026**

**JULIO 2026 , SAN FRANCISCO GOTERA, MORAZAN EL SALVADOR. C.A.**

# ENTREGABLE DE PROYECTO BACKEND

## SISTEMA DE RESERVACIONES DE MESAS - API REST PROFESIONAL

Node.js • Express • PostgreSQL / MySQL • JWT • Bcrypt • Swagger

**Resumen de Arquitectura:** Esta solución contiene una API REST modular en Node.js implementando buenas prácticas de desarrollo: arquitectura multicapa (Controladores, Servicios, Repositorios/Modelos), inyección segura mediante middlewares para JWT, roles de usuario, validaciones robustas de disponibilidad de mesas en un mismo bloque de tiempo y documentación auto-generada con Swagger.

### 1. Modelo de Base de Datos y Script SQL

A continuación se define el esquema relacional compatible con PostgreSQL / MySQL, asegurando llaves foráneas e índices para agilizar las búsquedas por fechas y rangos horarios:

-- Creación de la Base de Datos e Inserción Inicial (schema.sql)

```
CREATE TABLE roles (  
  id SERIAL PRIMARY KEY,  
  nombre VARCHAR(50) UNIQUE NOT NULL  
);  
  
CREATE TABLE usuarios (  
  id SERIAL PRIMARY KEY,  
  nombre VARCHAR(100) NOT NULL,  
  email VARCHAR(100) UNIQUE NOT NULL,  
  password VARCHAR(255) NOT NULL,  
  rol_id INT REFERENCES roles(id) DEFAULT 1,  
  fecha_creacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);  
  
CREATE TABLE mesas (  
  id SERIAL PRIMARY KEY,  
  numero_mesa INT UNIQUE NOT NULL,  
  capacidad INT NOT NULL,  
  descripcion VARCHAR(255),  
  estado VARCHAR(20) DEFAULT 'disponible' -- 'disponible', 'mantenimiento'  
);  
  
CREATE TABLE reservaciones (  
  id SERIAL PRIMARY KEY,  
  usuario_id INT REFERENCES usuarios(id) ON DELETE CASCADE,
```

```

mesa_id INT REFERENCES mesas(id) ON DELETE CASCADE,
fecha DATE NOT NULL,
hora TIME NOT NULL,
comensales INT NOT NULL,
estado VARCHAR(20) DEFAULT 'pendiente', -- 'pendiente', 'confirmada', 'cancelada' fecha_creacion
TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Índices optimizados para evitar duplicados en bloques de fecha y hora
CREATE INDEX idx_reservaciones_fecha_hora ON reservaciones(fecha, hora, mesa_id);

-- Inserción de Semillas (Seed data)
INSERT INTO roles (id, nombre) VALUES (1, 'Cliente'), (2, 'Admin') ON CONFLICT DO NOTHING;

-- Contraseñas cifradas para pruebas (ejemplo de bcrypt con salt): '123456'
-- Admin: admin@restaurante.com / Cliente: cliente@restaurante.com
INSERT INTO usuarios (nombre, email, password, rol_id) VALUES
('Admin Restaurante', 'admin@restaurante.com', '$2b$10$R9h8l.e8yS4vFhZHe7s0euXFW.m.SveH/Gf1oCOZ1F9O2yZIG9F2u', 2),
('Juan Pérez', 'cliente@restaurante.com', '$2b$10$R9h8l.e8yS4vFhZHe7s0euXFW.m.SveH/Gf1oCOZ1F9O2yZIG9F2u', 1);

INSERT INTO mesas (numero_mesa, capacidad, descripcion) VALUES
(1, 2, 'Mesa romántica junto a la ventana'),
(2, 4, 'Mesa familiar zona central'),
(3, 6, 'Mesa grande terraza exterior'),
(4, 2, 'Mesa barra de bar');

```

Para asegurar un código escalable, mantenible y legible, estructuramos el directorio de la siguiente manera:

```
restaurant-api/
```

| |— swagger/ # Configuración de la documentación interactiva Swagger | |— app.ts / app.js # Punto de entrada de la aplicación

|— .env.example # Plantilla de configuración de variables de entorno

|— package.json # Dependencias del proyecto

|— README.md # Manual de instalación, despliegue y uso de la API

### 3. Implementación de Códigos Clave del Backend

#### 3.1 Servidor Express Principal (app.js / src/app.js)

```
const express = require('express');
const cors = require('cors');
const morgan = require('morgan');
const swaggerUi = require('swagger-ui-express');
const swaggerSpec = require('./swagger/swaggerConfig');
require('dotenv').config();
const authRoutes = require('./routes/auth.routes');
const mesasRoutes = require('./routes/mesas.routes');
const reservacionesRoutes = require('./routes/reservaciones.routes');
const app = express();
// Middlewares globales
app.use(cors());
app.use(express.json());
app.use(morgan('dev'));
// Documentación de la API
app.use('/api-docs', swaggerUi.serve, swaggerUi.setup(swaggerSpec));
// Rutas de la API
app.use('/api/auth', authRoutes);
app.use('/api/mesas', mesasRoutes);
app.use('/api/reservaciones', reservacionesRoutes);
// Manejo de rutas inexistentes
app.use((req, res) => {
  res.status(404).json({ error: 'Ruta no encontrada' });
});
// Manejador global de errores
app.use((err, req, res, next) => {
  console.error(err.stack);
```

```

res.status(500).json({ error: 'Error interno del servidor' });
});
const PORT = process.env.PORT || 3000;
app.listen(PORT, () => {
  console.log(`Servidor ejecutándose en el puerto ${PORT}`);
  console.log(`Documentación interactiva disponible en http://localhost:${PORT}/api-docs`); });
module.exports = app;

```

**3.2 Middlewares de Seguridad y Control de Roles (src/middlewares/auth.middleware.js)**

```

const jwt =
require('jsonwebtoken');
const verificarToken = (req, res, next) => {
  const authHeader = req.headers['authorization'];
  const token = authHeader && authHeader.split(' ')[1];
  if (!token) {
    return res.status(401).json({ error: 'Acceso denegado. Token no proporcionado.' }); }
  try {
    const verificado = jwt.verify(token, process.env.JWT_SECRET || 'secreto_super_seguro_123'); req.usuario =
verificado; // Guarda el id y rol en la petición para uso posterior next();
  } catch (error) {
    return res.status(403).json({ error: 'Token inválido o expirado.' });
  }
};
const requerirRol = (rolesPermitidos) => {
  return (req, res, next) => {
    if (!req.usuario || !rolesPermitidos.includes(req.usuario.rol)) {
      return res.status(403).json({ error: 'No tienes permisos suficientes para realizar esta acción.' });
    }
    next();
  };
};
module.exports = { verificarToken, requerirRol };

```

### 3.3 Lógica de Negocio: Reservaciones con Validación de Disponibilidad Temporal

El núcleo de la actividad de evaluación radica en que un cliente no pueda reservar una mesa si esta se encuentra ocupada en el mismo bloque temporal. A continuación, el controlador encargado de resolver esta regla de negocio de manera segura:

```
const db = require('../config/db'); // Módulo de conexión a la Base de Datos

exports.crearReservacion = async (req, res) => {
  const { mesa_id, fecha, hora, comensales } = req.body;

  const usuario_id = req.usuario.id; // Obtenido de forma segura desde el JWT
  if (!mesa_id || !fecha || !hora || !comensales) {
    return res.status(400).json({ error: 'Todos los campos son obligatorios.' }); }

  try {
    // 1. Validar que la mesa exista y esté activa
    const mesaQuery = 'SELECT * FROM mesas WHERE id = $1';
    const { rows: mesas } = await db.query(mesaQuery, [mesa_id]);

    if (mesas.length === 0) {
      return res.status(404).json({ error: 'La mesa seleccionada no existe.' }); }

    if (mesas[0].estado !== 'disponible') {
      return res.status(400).json({ error: 'La mesa seleccionada se encuentra en mantenimiento.' }); }

    if (comensales > mesas[0].capacidad) {
      return res.status(400).json({ error: `La mesa tiene una capacidad máxima de $ {mesas[0].capacidad} comensales.` }); }
  }

  // 2. REGLA CRÍTICA: Validar que la mesa no esté reservada en el mismo bloque de fecha y hora // Se
  // asume un bloque estándar de reservación (ej. intervalos de 1 hora o coincidencia exacta de bloques)
  const reservacionExistenteQuery = `
  SELECT * FROM reservaciones
  WHERE mesa_id = $1
  AND fecha = $2
  AND hora = $3
  AND estado != 'cancelada'
  `;

  const { rows: colisiones } = await db.query(reservacionExistenteQuery, [mesa_id, fecha, hora]); if
  (colisiones.length > 0) {
    return res.status(409).json({
```

```

error: 'La mesa no está disponible en la fecha y hora seleccionadas.' });
}
// 3. Crear reservación
const insertarQuery = `
INSERT INTO reservaciones (usuario_id, mesa_id, fecha, hora, comensales, estado) VALUES ($1, $2, $3, $4,
$5, 'pendiente')
RETURNING *
`;
const { rows: nuevaReservacion } = await db.query(insertarQuery, [usuario_id, mesa_id, fecha, hora,
comensales]);
return res.status(211).json({
mensaje: 'Reservación creada con éxito',
reservacion: nuevaReservacion[0]
});
} catch (error) {
console.error(error);
return res.status(500).json({ error: 'Error interno del servidor al crear la reservación.' }); }
};

```

#### 4. Guía para Despliegue Exitoso (Railway / Render)

Para asegurar que la entrega de la **API en Producción** se realice sin inconvenientes, siga estos pasos: 1.

**Variables de entorno:** Configure la variable DATABASE\_URL (con la cadena de conexión provista por Render o Railway), JWT\_SECRET con un hash robusto, y PORT=8080.

2.

**Scripts en Package.json:** Defina obligatoriamente "start": "node src/app.js" para que la plataforma de hosting sepa exactamente cómo ejecutar la aplicación una vez descargadas las dependencias con npm install.